

Component Based or Monolithic Development for Large C and C++ Projects

Why Not Both?

DIEGO RODRIGUEZ-LOSADA



CONAN 2.0
C/C++ Package Manager

Development paradigms

Mono-repo / monolithic build

git@.../monorepo.git

```
| -WORKSPACE
| -liba
|   | -BUILD
|   | -include
|   |   | -a.h
|   | -src
|   |   | -a.cpp
| -libb
|   | -BUILD
|   | -include
|   |   | -b.h
|   | -src
|   |   | -b.cpp
| -app1
|   | -BUILD
|   | -src
|   |   | -main.cpp
```

Multi-repo / component build

git@.../liba.git

```
| -liba
|   | -CMakeList.txt
|   | -include
|   |   | -a.h
|   | -src
|   |   | -a.cpp
```

git@.../libb.git

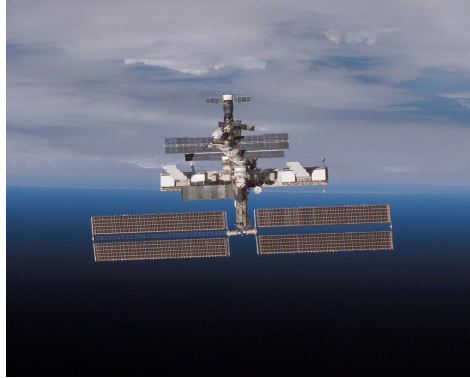
```
| -libb
|   | -CMakeList.txt
|   | -include
|   |   | -b.h
|   | -src
|   |   | -b.cpp
```

git@.../app1.git

```
| -app1
|   | -CMakeList.txt
|   | -src
|   |   | -main.cpp
```

Component based paradigm

Seen by component
based
developers



Seen by monorepo
based
developers



Monorepo based paradigm



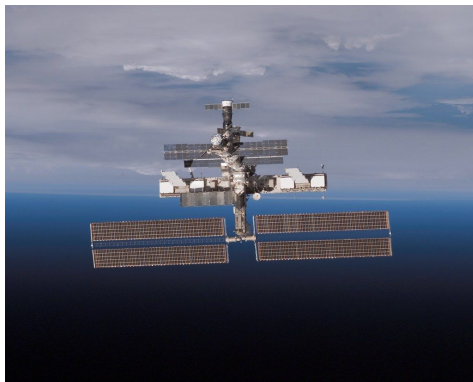
Component based paradigm

Seen by component
based
developers



Monorepo based paradigm

Seen by monorepo
based
developers



Conway's law

Organizations which design systems (in the broad sense used here) are constrained to produce designs which are copies of the communication structures of these organizations.

—Melvin E. Conway, How Do Committees Invent?

The structure of any system designed by an organization is isomorphic to the structure of the organization

You can see the organization chart of a car company in the dashboard, and also see whether the steering wheel team hates the gear stick team.

Mono repo

```
| -WORKSPACE
| -liba
|   | -BUILD
|   | -include
|   |   | -a.h
|   | -src
|   |   | -a.cpp
| -libb
|   | -BUILD
|   | -include
|   |   | -b.h
|   | -src
|   |   | -b.cpp
| -app1
|   | -BUILD
|   | -src
|   |   | -main.cpp
```

- Tooling:
 - Bazel(blaze), Buck2, Visual
 - Build cache + heavy infra + custom tooling
- Pros:
 - Live at head: No versioning
- Cons:
 - No versioning
 - Organizational challenges
 - Infra



Bazel



Multi-repo/components

- Classic versioning paradigm
- Tooling:
 - CMake, Makefiles, MSBuild, Meson
 - Binary cache (pkg management)
- Pros:
 - Per component development, versioning and releasing
- Cons:
 - Per component development, versioning and releasing



git@.../liba.git

```
| -liba
|   | -CMakeList.txt
|   | -include
|   |   | -a.h
|   | -src
|       | -a.cpp
```

git@.../libb.git

```
| -libb
|   | -CMakeList.txt
|   | -include
|   |   | -b.h
|   | -src
|       | -b.cpp
```

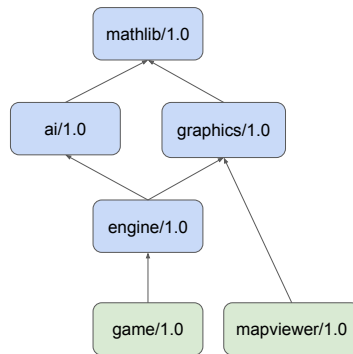
git@.../app1.git

```
| -app1
|   | -CMakeList.txt
|   | -src
|       | -main.cpp
```



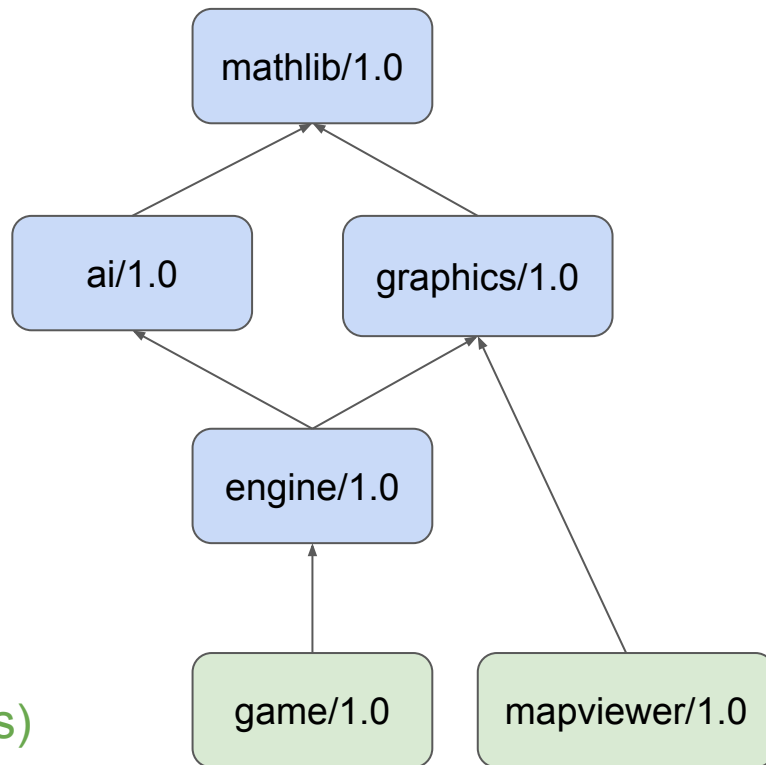
Outline

- Intro
- **Example project**
 - **Package management basics**
- The challenge: evolving packages
- CI at scale problem
- Workspace problem
 - The orchestrated workspace
 - The mono-repo/super-build workspace
 - The dynamic workspace
- Conclusions



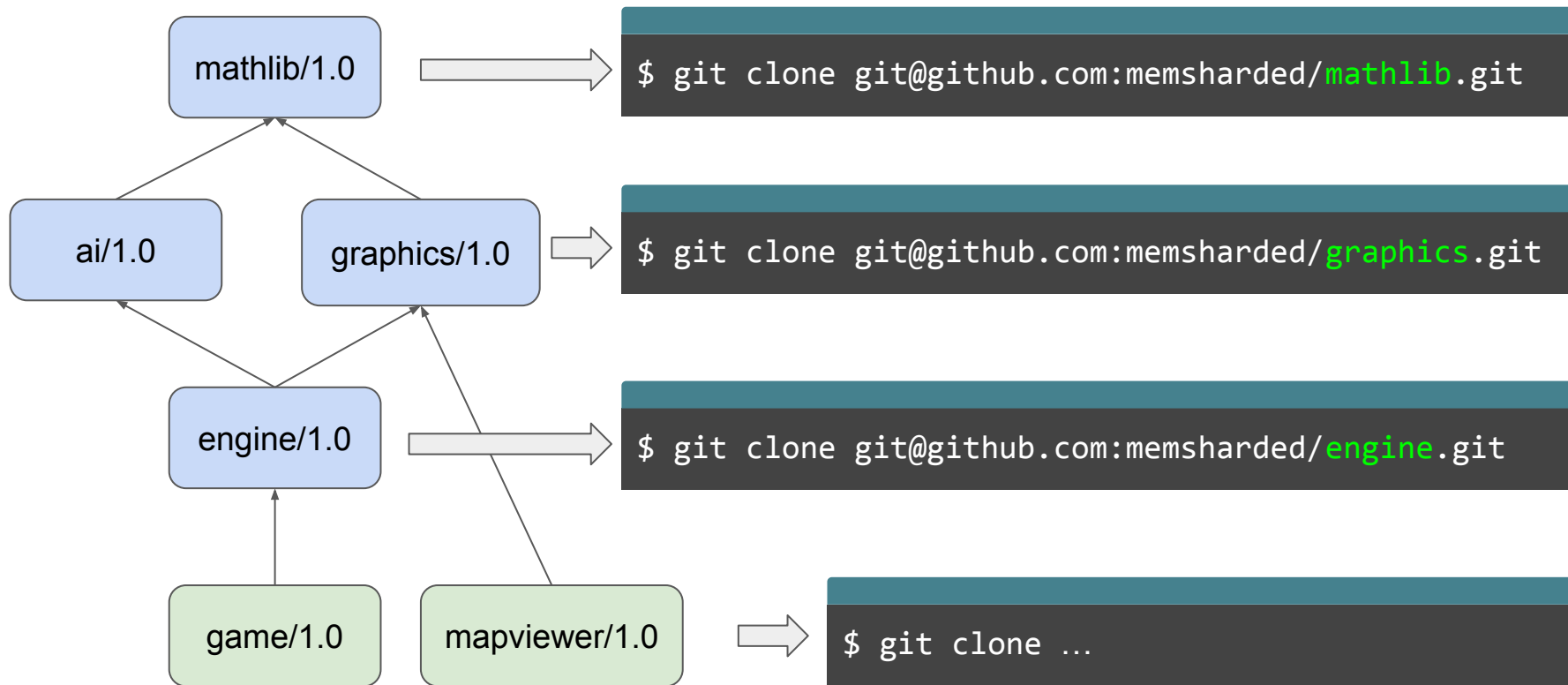
Example project

Libraries (static)

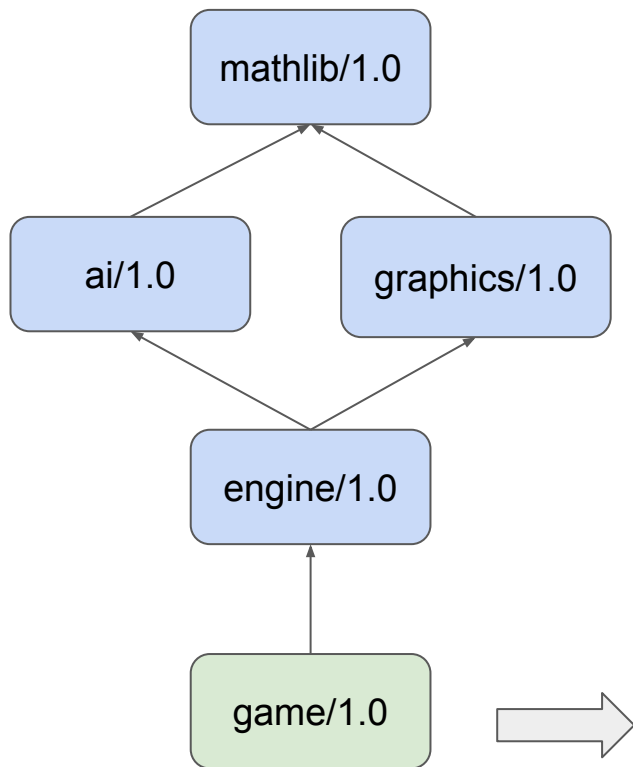


Applications (exes)

Example project: multi-repository

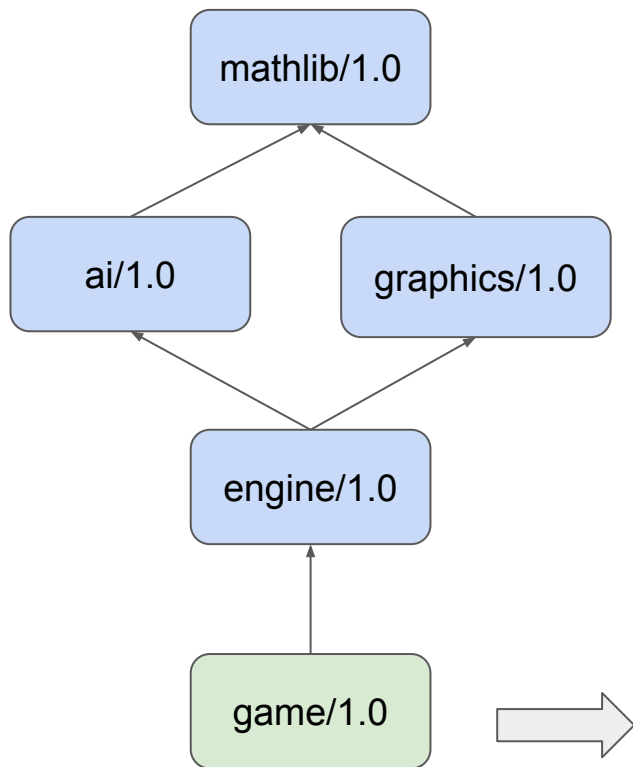


Example project: multi-repository



```
$ game.exe
mathlib/1.0: mathlib maths!
ai/1.0: Some Artificial Intelligence for aliens!
graphics/1.0: Checking if things collide!
engine/1.0: Computing some game things!
game/1.0: fun game!
```

Example project: building & package management 101



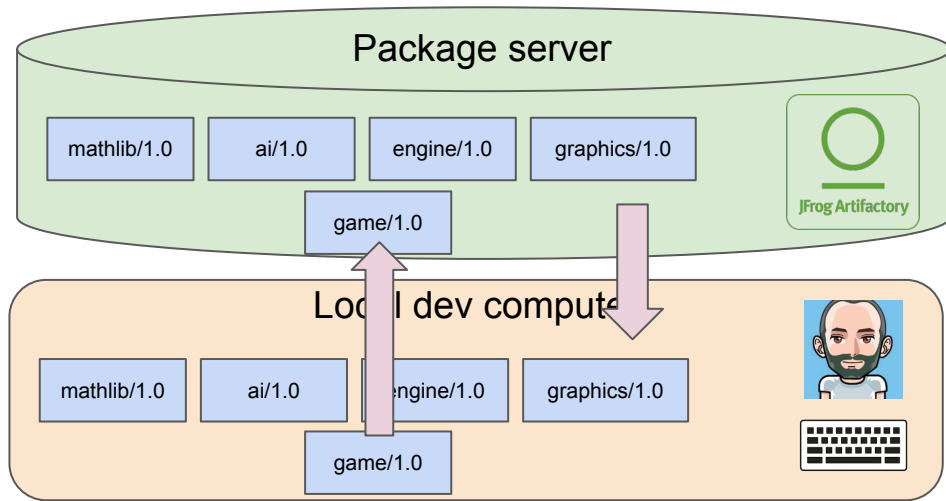
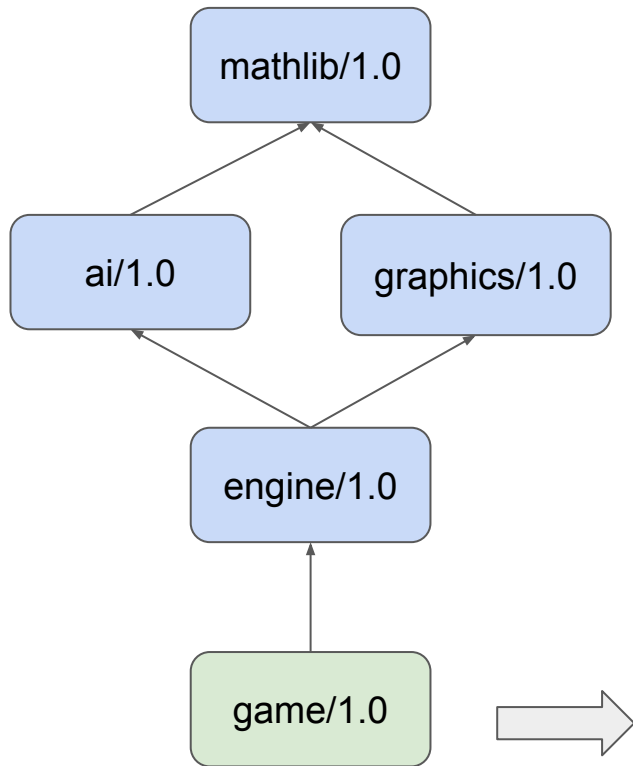
```
$ conan install
```

```
$ conan create
```

```
$ conan upload
```

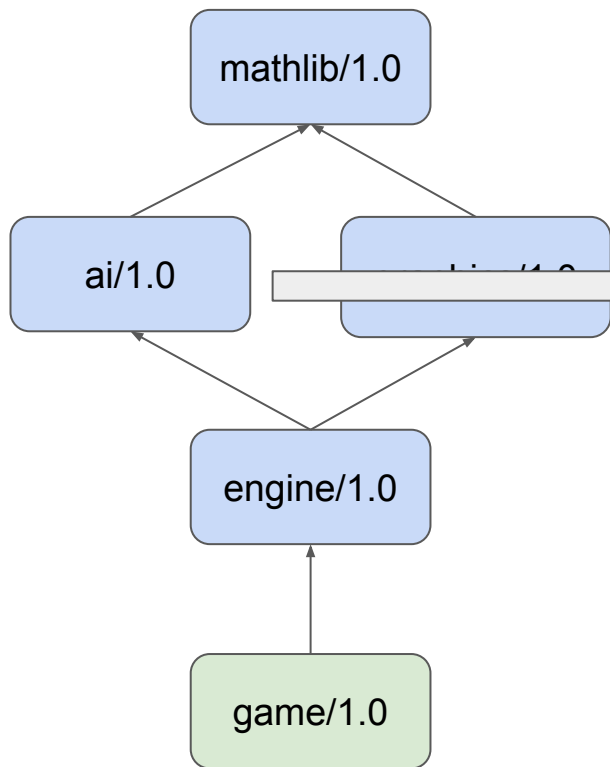
```
$ git clone git@github:memsharded/game && cd game  
$ conan install  
$ cmake --preset conan-default  
# build, edit, work  
$ conan create  
$ conan upload -r=myremote
```

Example project: building & package management 101



```
$ git clone git@github:memsharded/game && cd game
$ conan install
$ cmake --preset conan-default
# build, edit, work
$ conan create
$ conan upload -r=myremote
```

Example project: building & package management 101



```
$ git clone git@github:memsharded/ai && cd ai  
$ conan install # only mathlib/1.0!  
$ cmake --preset conan-default  
# build, edit, work
```

conanfile.py

```
class aiRecipe(ConanFile):
    name = "ai"
    version = "1.0"
    requires = "mathlib/[>=1.0 <2]"

    # Binary configuration
    settings = "os", "compiler", "build_type", "arch"
    package_type = "static-library"

    def export(self):
        git = Git(self, self.recipe_folder)
        git.coordinates_to_conandata()

    def generate(self):
        tc = CMakeToolchain(self)
        tc.preprocessor_definitions["PKG_VERSION"] = f'"{self.version}"'
        tc.generate()
        deps = CMakeDeps(self)
        deps.generate()

    def build(self):
        cmake = CMake(self)
        cmake.configure()
        cmake.build()

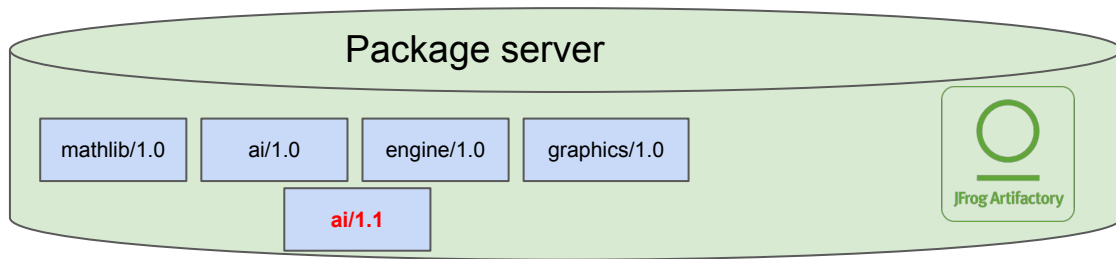
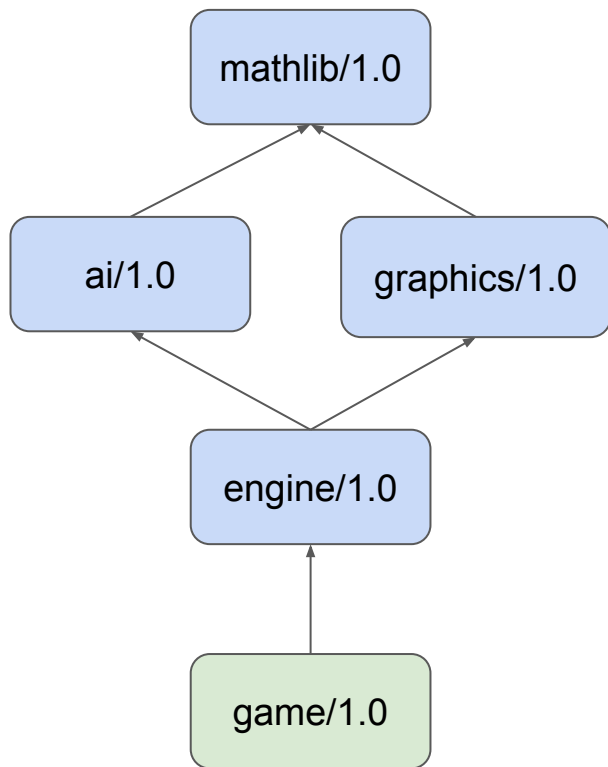
    def package(self):
        cmake = CMake(self)
        cmake.install()

    def package_info(self):
        self.cpp_info.libs = ["ai"]
```

Outline

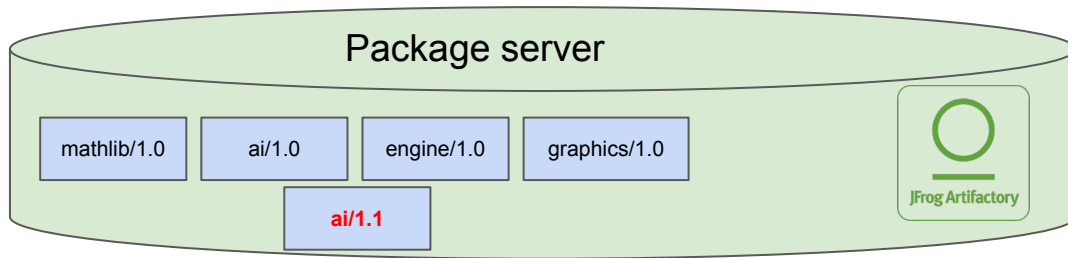
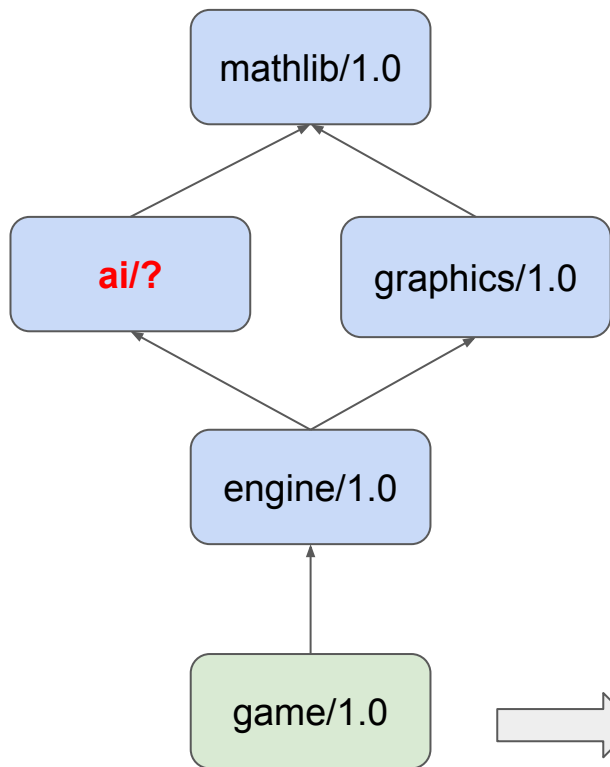
- Intro
- Example project
 - Package management basics
- **The challenge: evolving packages**
- CI at scale problem
- Workspace problem
 - Definition
 - The orchestrated workspace
 - The mono-repo/super-build workspace
 - The dynamic workspace
- Conclusions

The challenge: evolving code



```
$ git clone git@github:memsharded/ai && cd ai
# changes code and bumps version
$ conan create # creates ai/1.1
$ conan upload -r=myremote
```

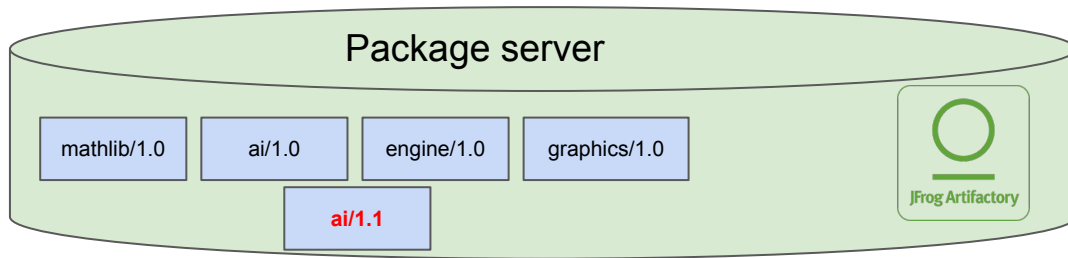
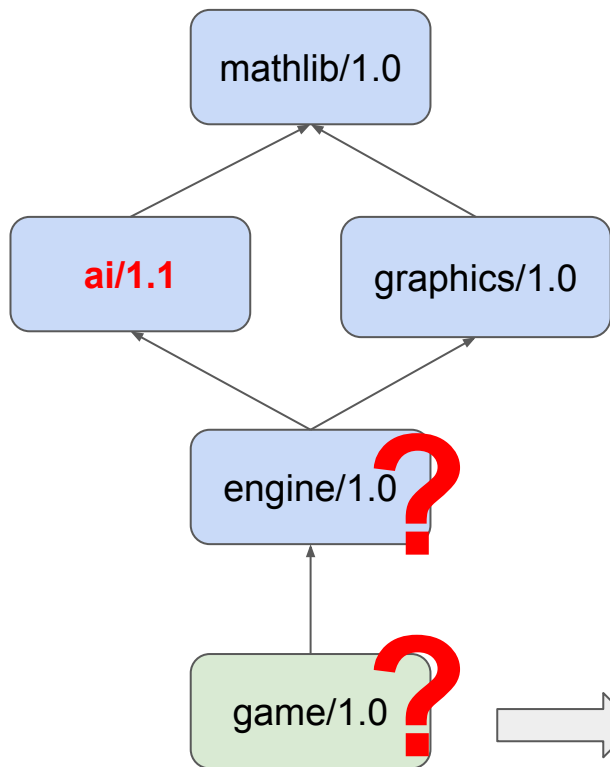
The challenge: evolving code



```
class EngineRecipe(ConanFile):  
    name = "engine"  
    version = "1.0"  
    requires = "ai/1.0", "graphics/1.0"  
    # OR  
    requires = "ai/[*]=1.0 <2]", "graphics/[*]=1.0 <2]"
```

```
$ git clone git@github:memsharded/game && cd game  
$ conan install
```

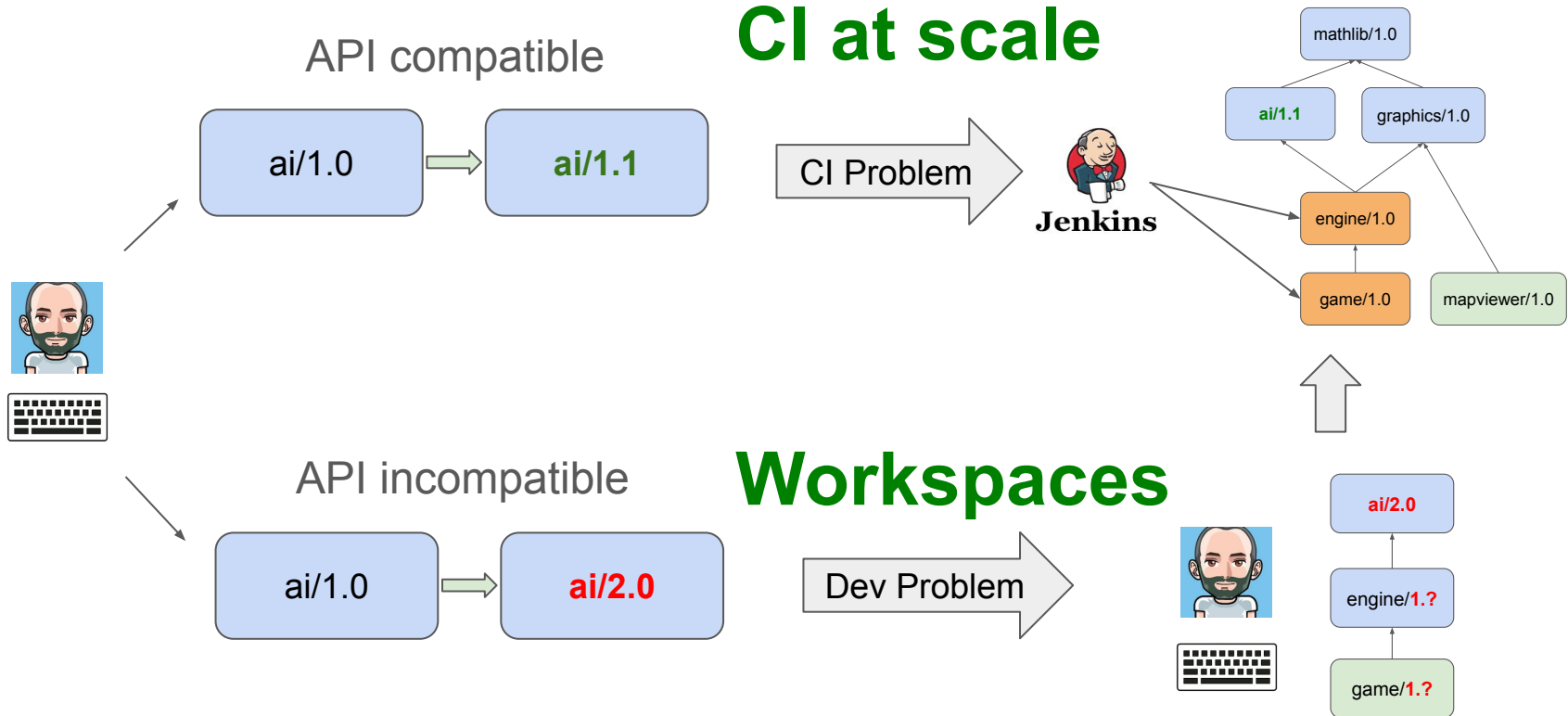
The challenge: evolving code



```
class EngineRecipe(ConanFile):  
    name = "engine"  
    version = "1.0"  
    requires = "ai/[>=1.0 <2]", "graphics/[>=1.0 <2]"
```

```
$ git clone git@github:memsharded/game && cd game  
$ conan install
```

Two different scenarios



Outline

- Intro
- Example project
 - Package management basics
- The challenge: evolving packages
- **CI at scale problem**
- Workspace problem
 - Definition
 - The orchestrated workspace
 - The mono-repo/super-build workspace
 - The dynamic workspace
- Conclusions



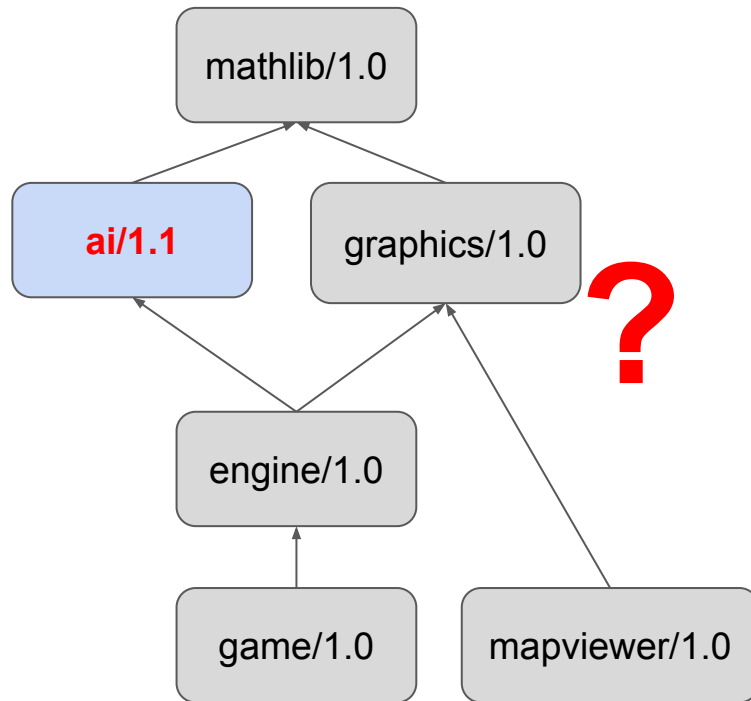
CI Problem statement

Given an API compatible new version of a package:

- Build and test the necessary packages for the supported platforms, in the right order

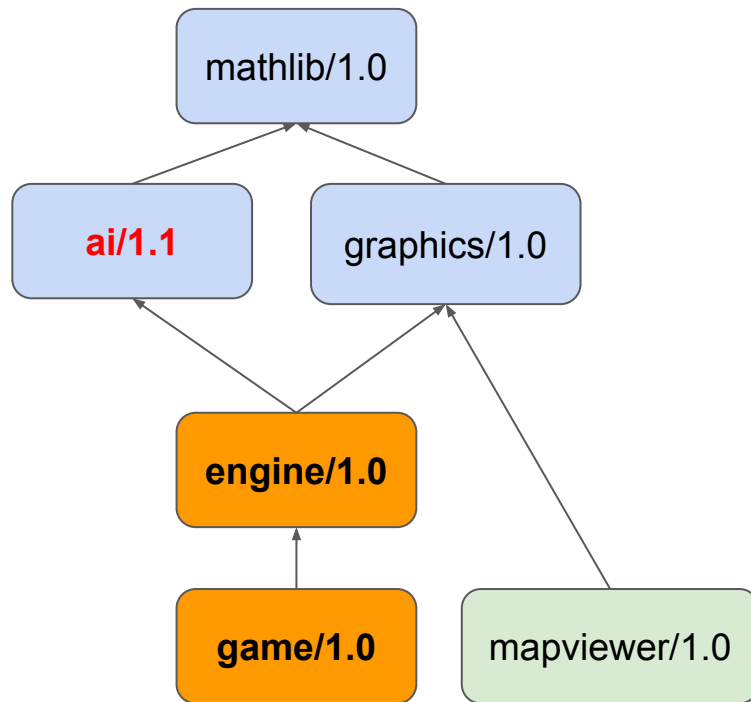
Conditions:

- **Efficient**: do not build more than necessary
- **Fast**: build in parallel whenever possible (distributed)
- **Safe**: do not break the build or disrupt other development and release processes
- **Maintainable**: don't duplicate the graph in CI



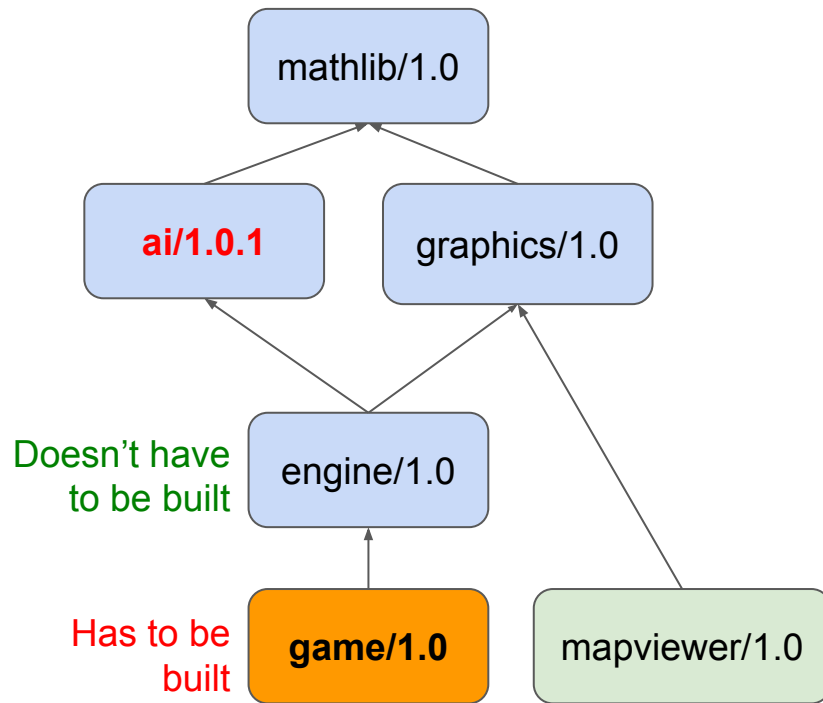
CI: Efficient

- Graph topology
- Versioning and package types



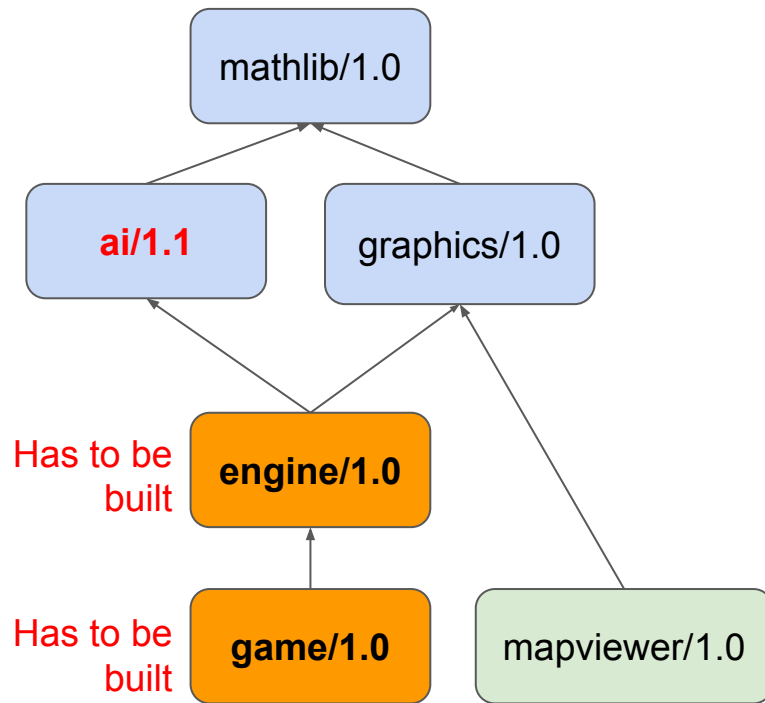
CI: Efficient

- Graph topology
- Versioning and package types
 - Patch **ai/1.0.1**
 - Meaning changes only to .cpp files
 - And “ai” and “engine” are static libraries
 - Only “game” has to be built



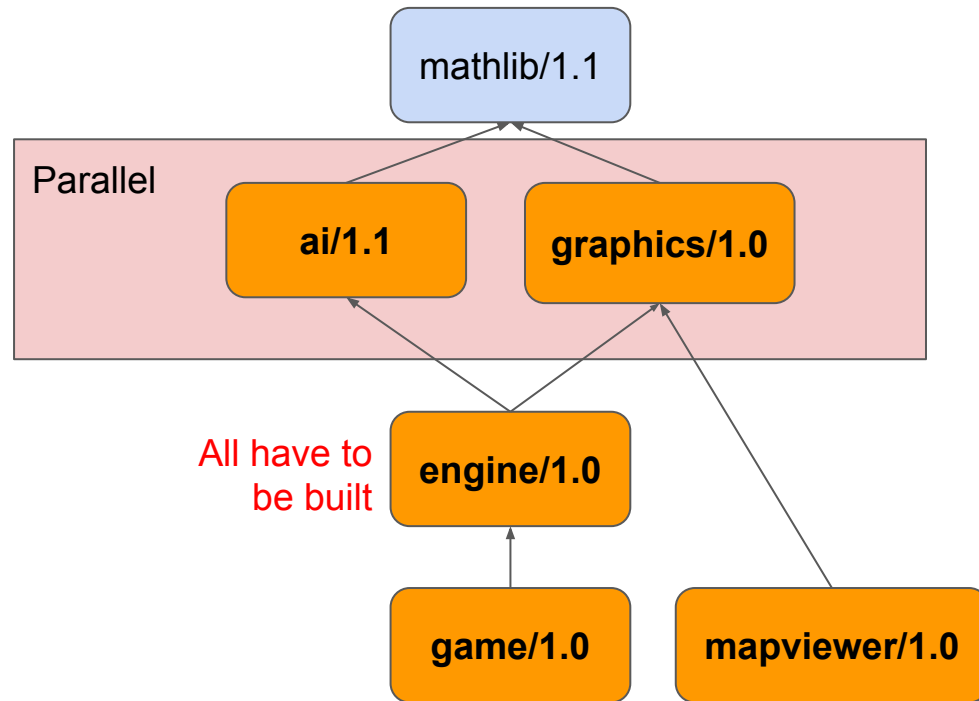
CI: Efficient

- Graph topology
- Versioning and package types
 - Patch **ai/1.0.1**
 - Meaning changes only to .cpp files
 - And “ai” and “engine” are static libraries
 - Only “game” has to be built
 - Minor **ai/1.1**
 - Meaning API compatible changes to .h headers too
 - “Engine” and “game” have to be built

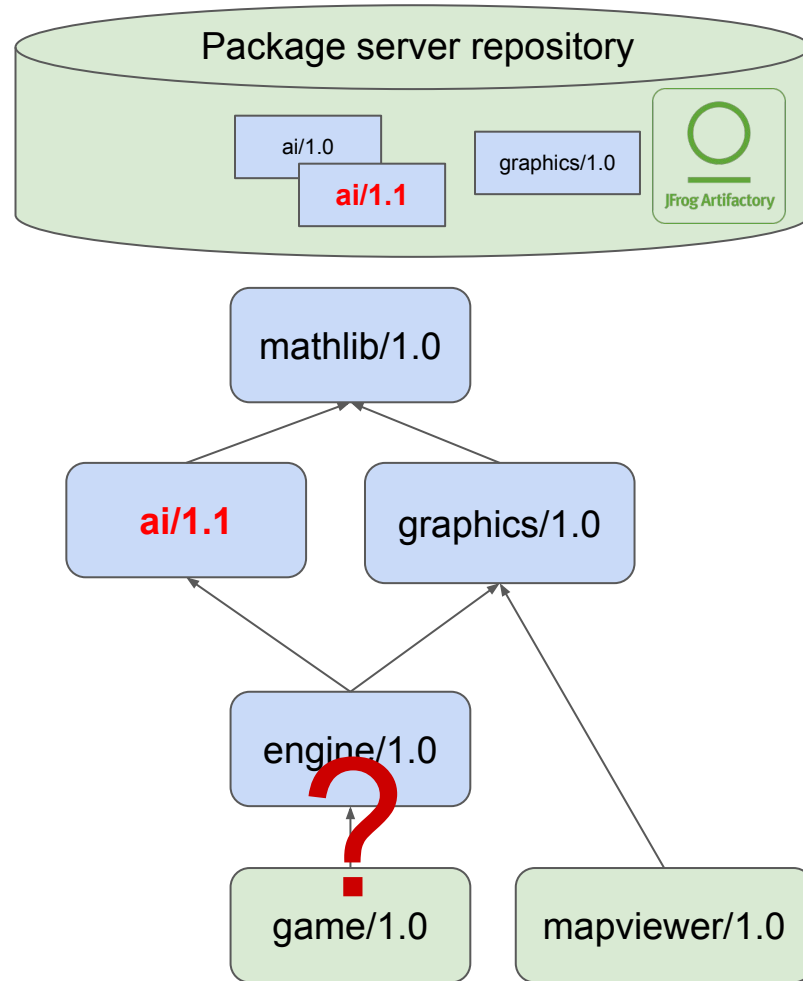


CI: Fast

- Exploit graph parallelism: Graph topology

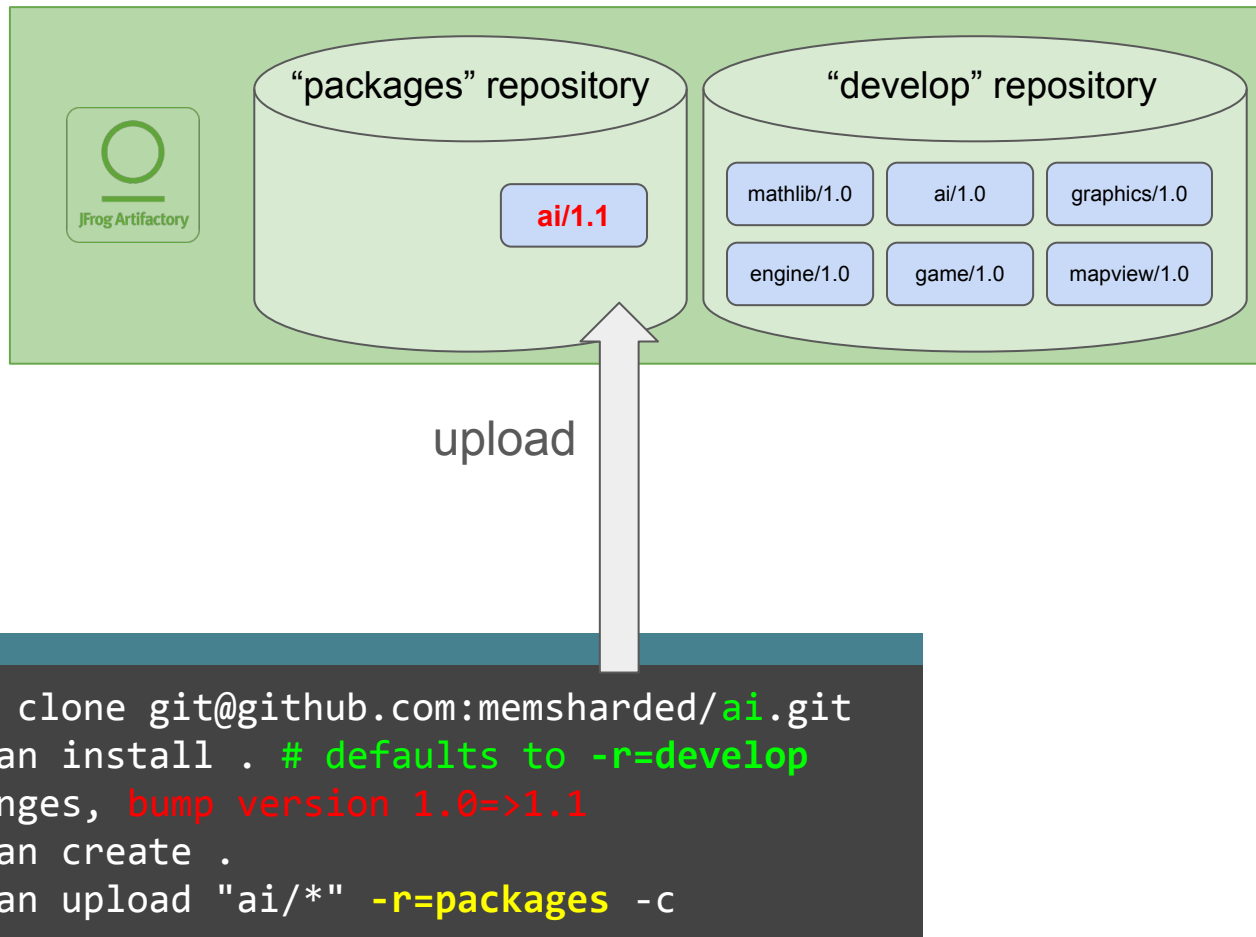


CI: Safe



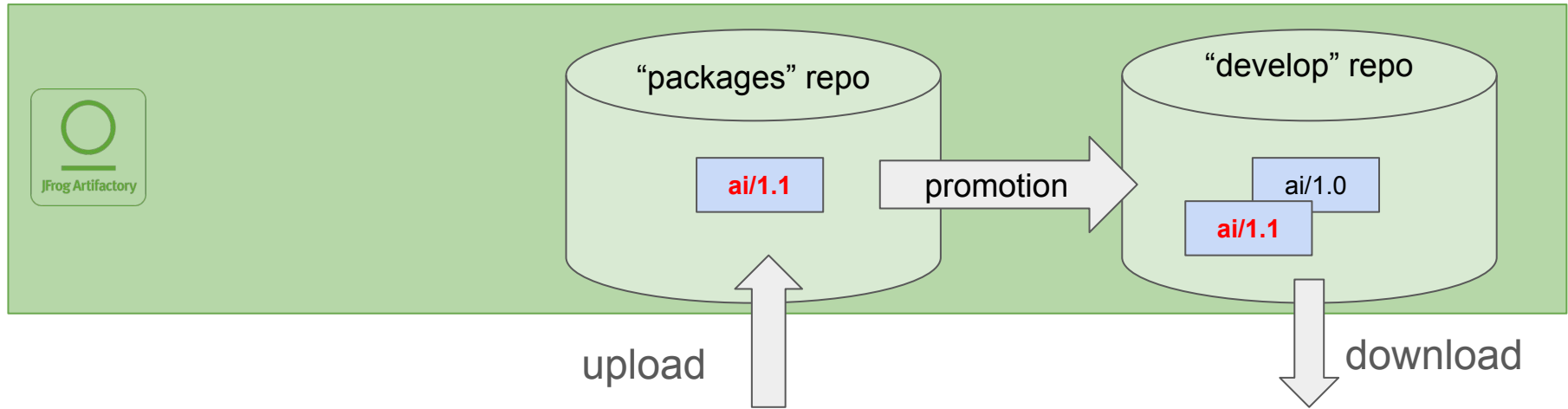
CI: Safe

Multi-repo in
package
management
↔ multi-branch
in source

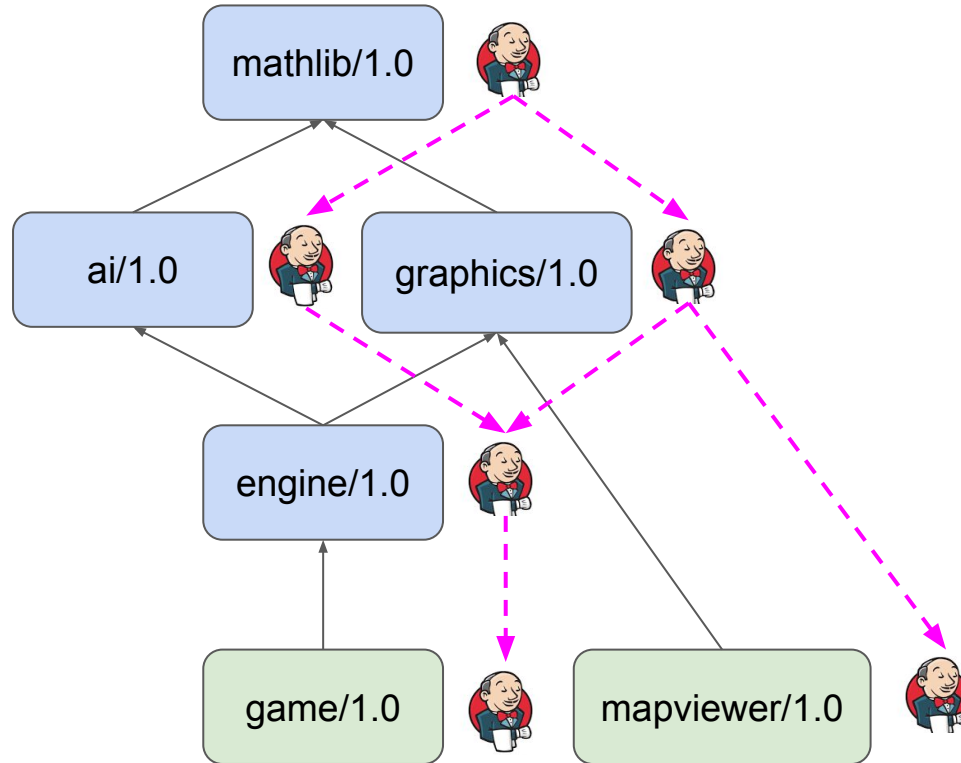


CI: Safe

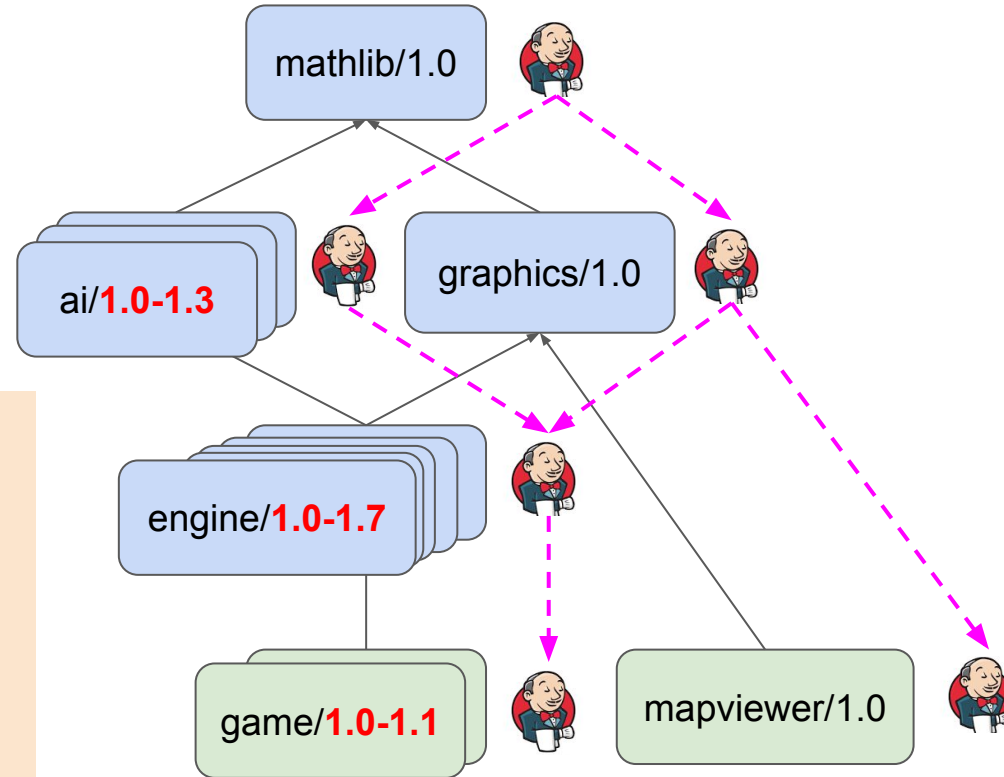
Promotions in package management $\Leftrightarrow$ merge in source



CI: Maintainable



CI: Maintainable?



Downstream
(consumers)
approach
doesn't scale

CI: Maintainable: “Products” approach

Step 1: Pick the
“products” of your
organization

- Business value
- Deliverables

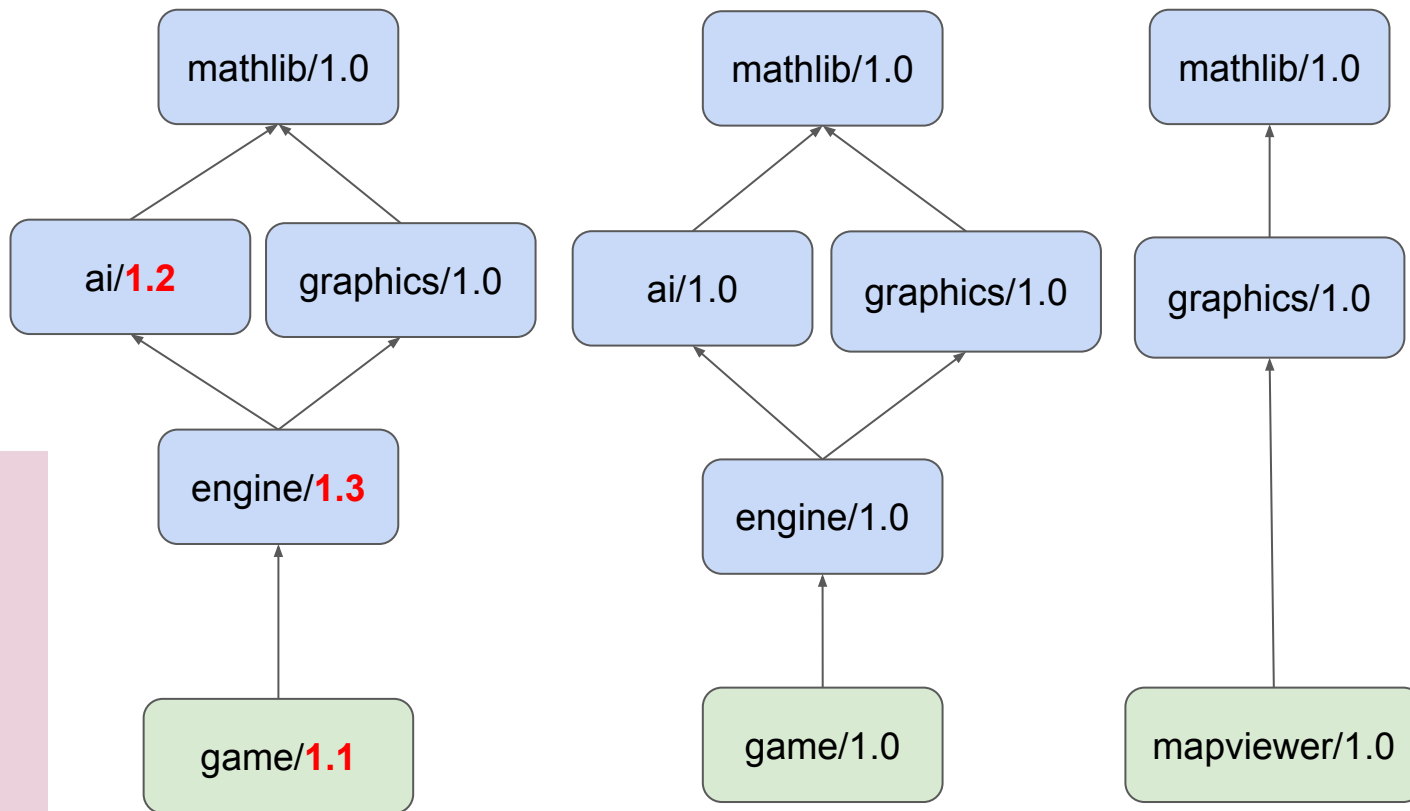


game/1.1

game/1.0

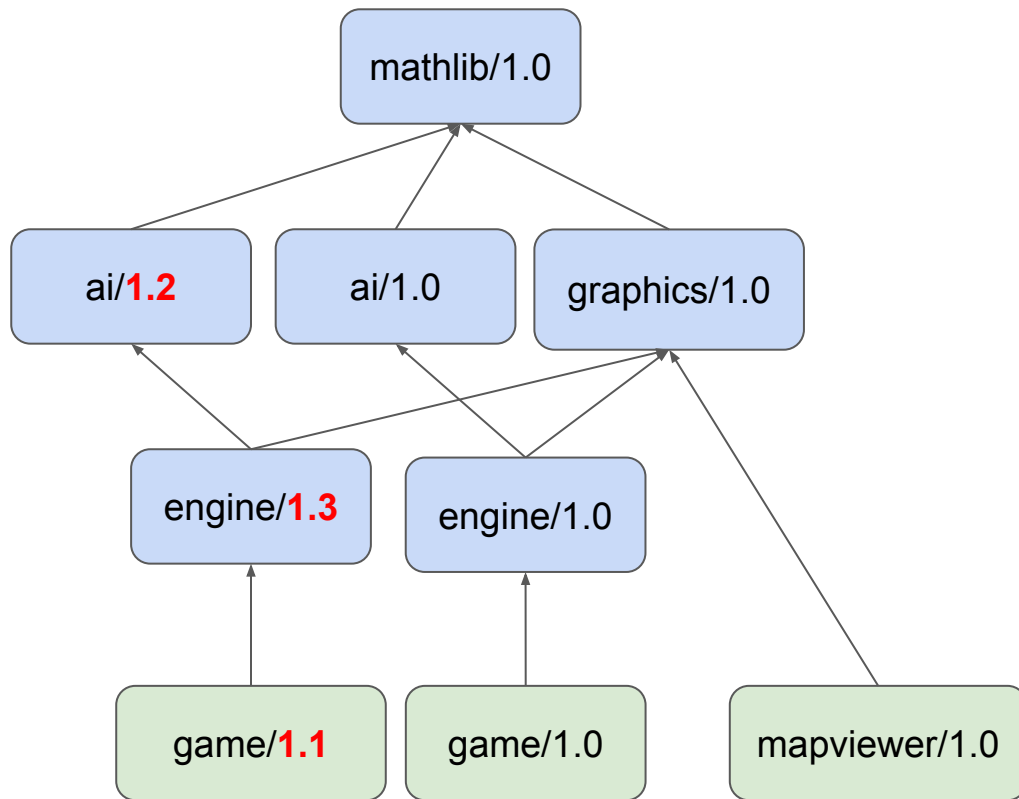
mapviewer/1.0

CI: Maintainable: “Products” approach



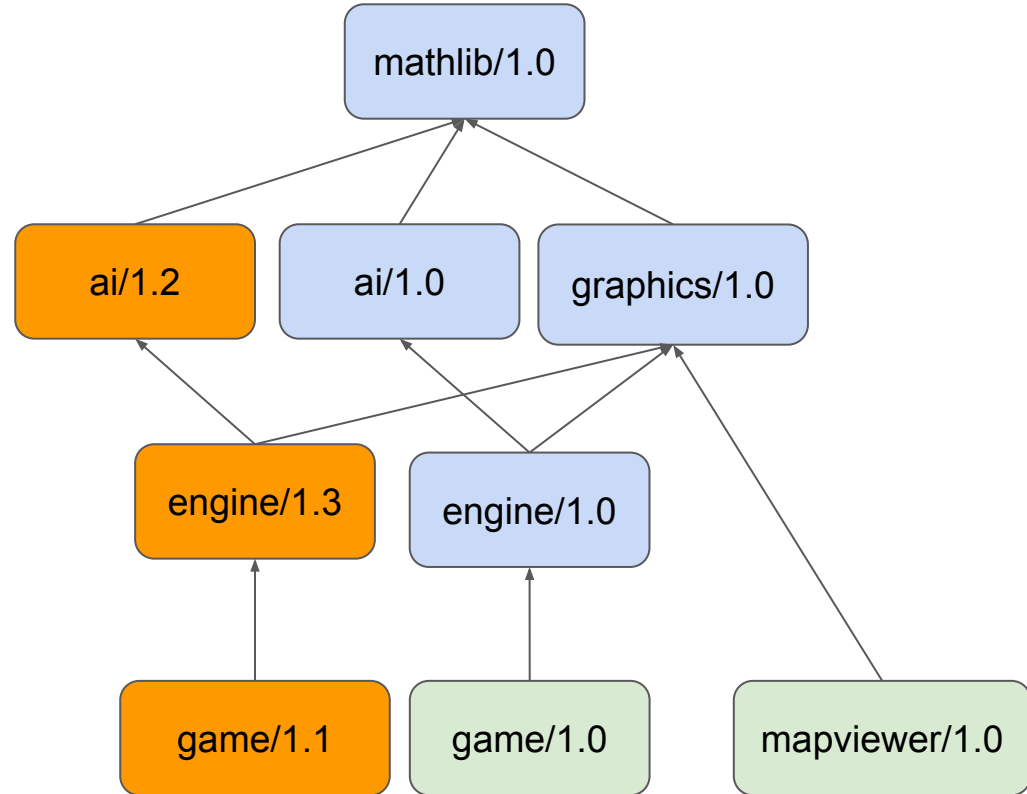
Step 2: Compute the dependency graphs of each product (consistently $\Leftrightarrow$ lockfiles)

CI: Maintainable: “Products” approach



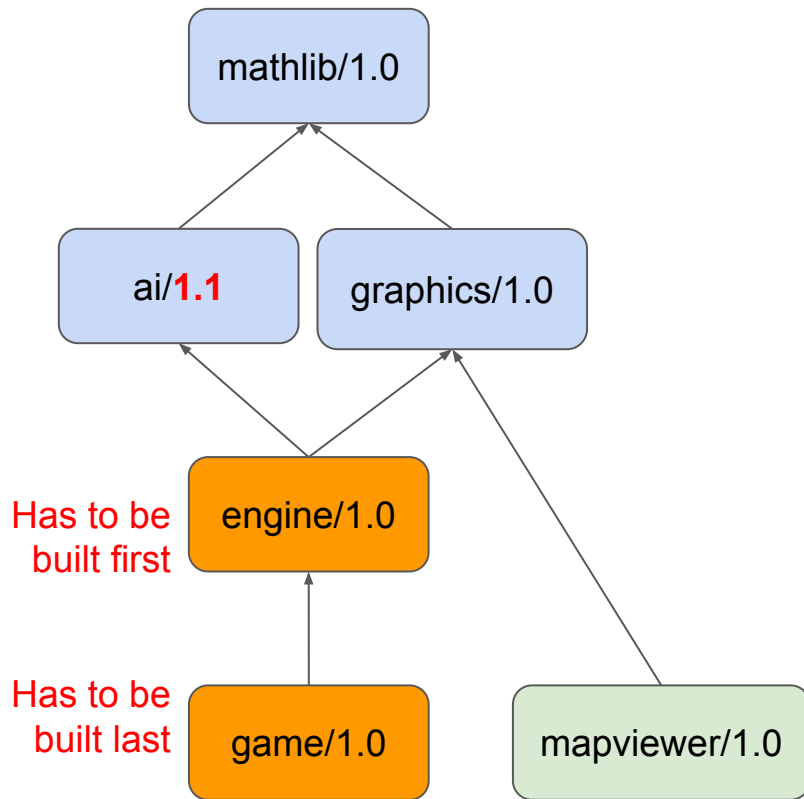
Step 3: Merge the graphs, keep topology

CI: Maintainable: “Products” approach



Step 4: Check what needs to be built, return as ordered groups

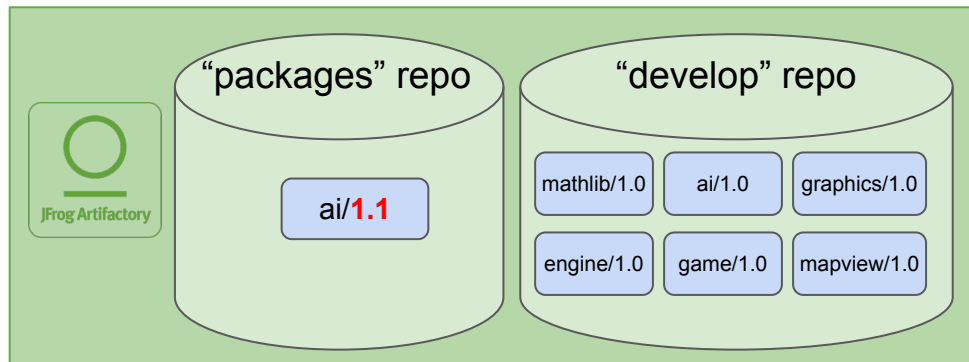
CI: Maintainable: “Products” approach



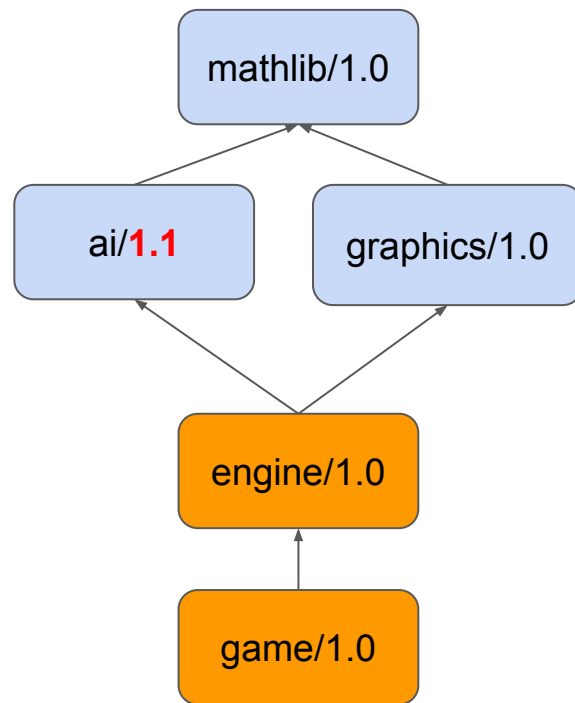
In our example before.

IMPORTANT:
New binaries for
same versions

Welcome “conan graph build-order”



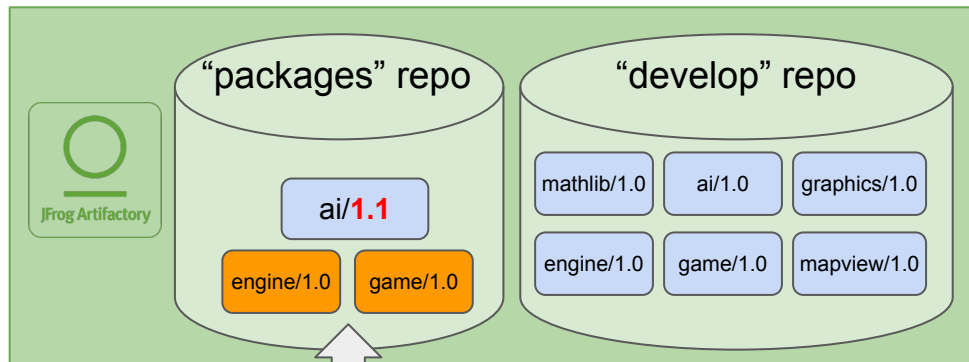
```
$ conan graph build-order -r packages  
-r develop --requires=game/1.0  
--build=missing > game_build_order.json
```



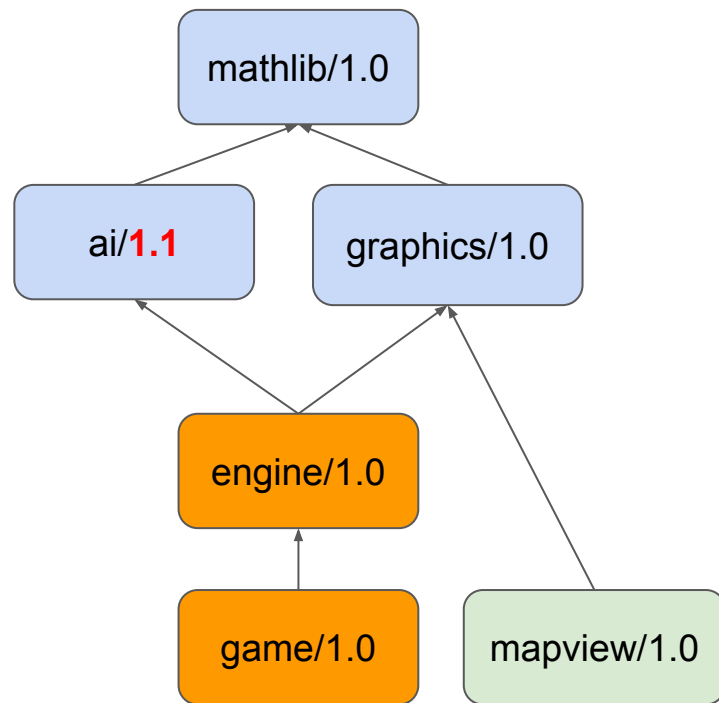
graph_build_order.json

```
[
  {
    "ref": "engine/1.0",
    "packages": [[{
      "package_id": "de73..a765",
      "binary": "Build",
      "build_args": "--requires=engine/1.0 --build=engine/1.0",
    }]]
  },
  [
    {
      "ref": "game/1.0",
      "depends": ["engine/1.0"],
      "packages": [[{
        "package_id": "bac7..9d4c",
        "binary": "Build",
        "build_args": "--requires=game/1.0 --build=game/1.0",
      }]]
    }
  ]
]
```

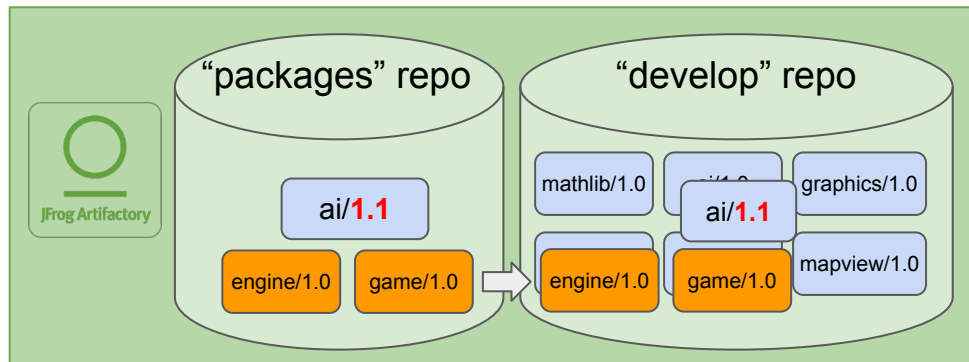
Build, upload and promote



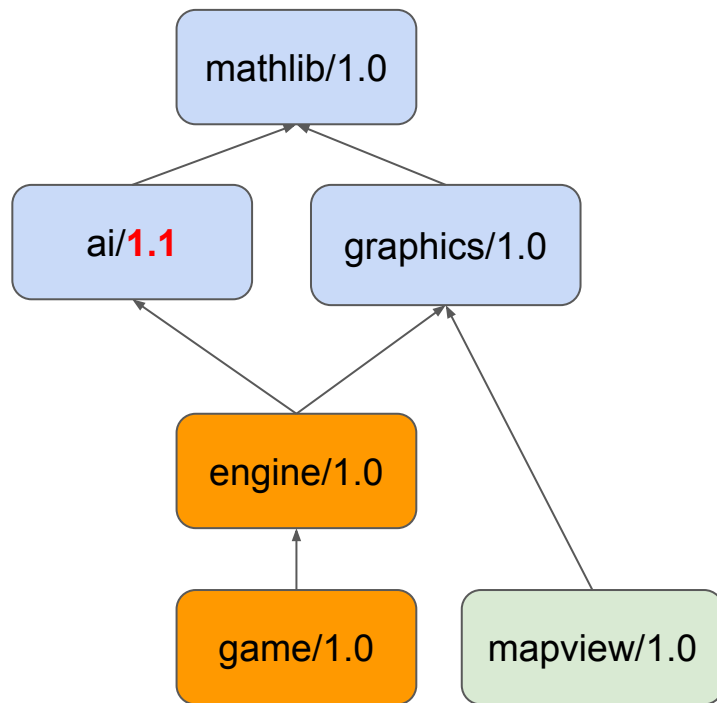
```
$ conan install --requires=engine/1.0  
--build=engine/1.0  
$ conan install --requires=game/1.0  
--build=game/1.0  
  
$ conan upload -r=packages
```



Build, upload and promote



```
$ conan install --requires=engine/1.0  
--build=engine/1.0  
$ conan install --requires=game/1.0  
--build=game/1.0  
  
$ conan upload -r=packages  
  
# promote packages => develop
```



Continuous Integration (CI) for Large Scale Package-Based C, C++ Projects With Conan2

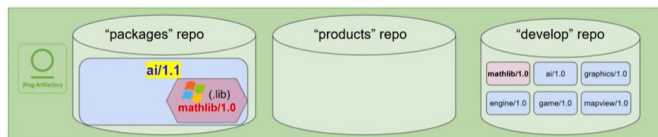
Diego Rodriguez-Losada

CONTINUOUS INTEGRATION (CI) FOR LARGE SCALE PACKAGE-BASED C, C++ PROJECTS WITH CONAN2

Bloomberg

ACCU
conference
2025

Building ai/1.1 for Linux



```
$ git clone git@<url>/ai.git && cd ai  
$ conan create . -pr=linux  
$ conan upload "ai/*" -r=packages
```



37

Diego Rodriguez-Losada

ACCU.ORG

<https://youtu.be/A3X1MpvYTrM>

https://docs.conan.io/2/ci_tutorial/tutorial.html

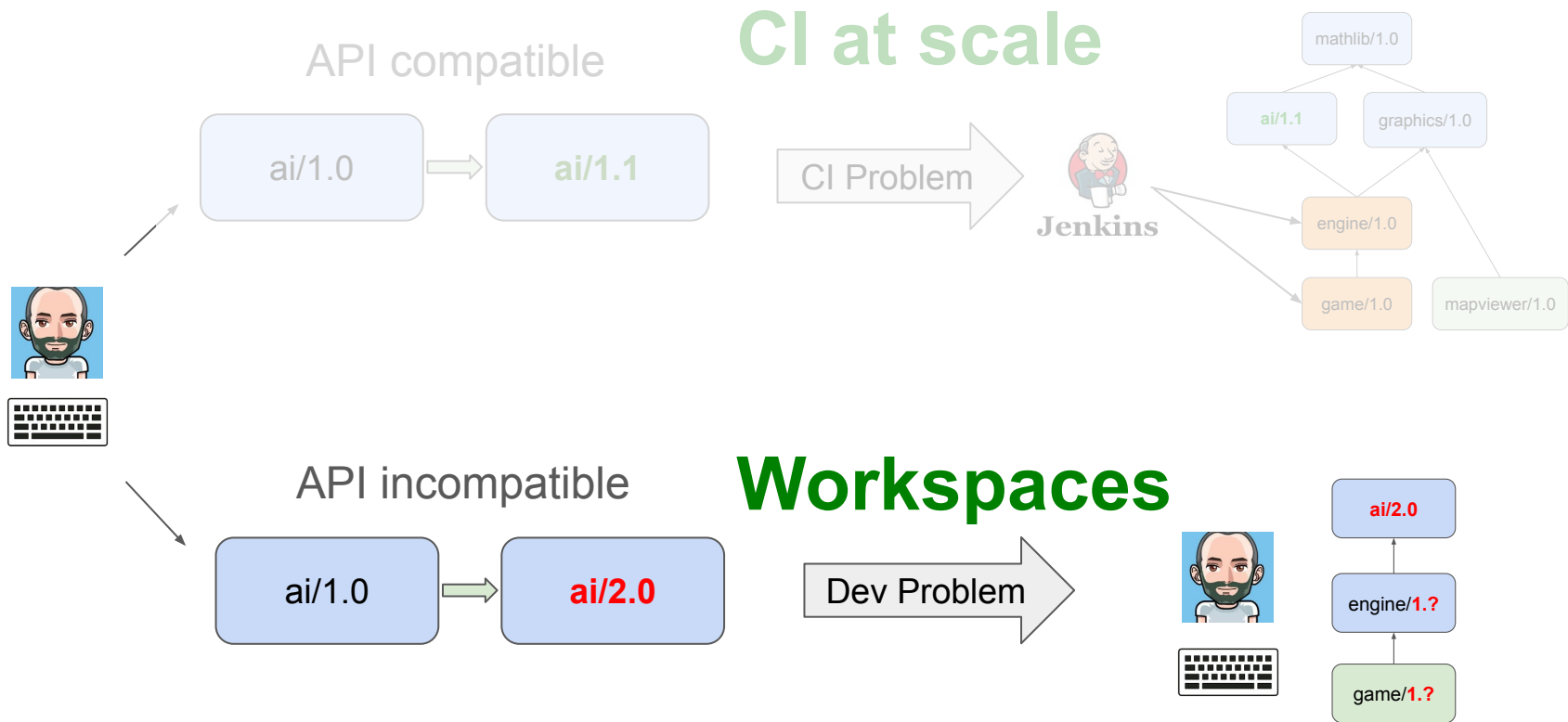
CI at scale (efficient, safe, fast, maintainable)

- Principles
 - Multi-repository (branches) and package promotions (merges)
 - Product pipelines (bottom-up approach)
 - Dependency graph management tools
- Implementation with Conan2
 - **200 lines of GH actions** code: **simple!**
 - No extra scripting necessary
 - **Escalable**, for any graph size, any number of configurations (architectures, platforms), any number or products. **Without explicit model in CI!**
 - Jenkins or similar preferred for the products pipeline
 - **Proven**: Learned from users

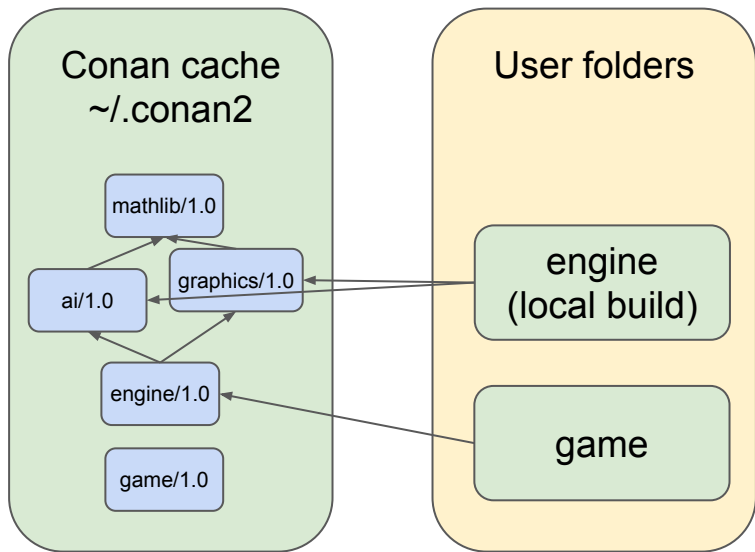
Outline

- Intro
- Example project
 - Package management basics
- The challenge: evolving packages
- CI at scale problem
- **Workspace problem**
 - The orchestrated workspace
 - The mono-repo/super-build workspace
 - The dynamic workspace
- Conclusions

The developer workspace problem



Working on multiple packages simultaneously

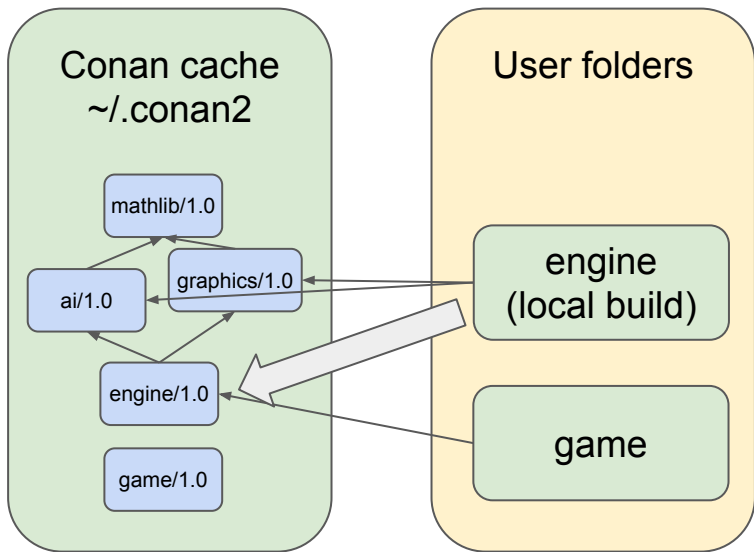


```
$ git clone git@.../engine.git
$ conan install
Requires
  mathlib/1.0 - Cache
  ai/1.1 - Cache
$ cmake --build # local build
```

```
$ git clone git@.../game.git
$ conan install
Requires
  mathlib/1.0 - Cache
  ai/1.0 - Cache
  engine/1.1 - Cache
```



Needs sharing via the package storage/cache



```
$ git clone git@.../engine.git
$ conan install
Requires
  mathlib/1.0 - Cache
  ai/1.1 - Cache
$ cmake --build # local build
$ conan create # full build
```

```
$ git clone git@.../game.git
$ conan install
Requires
  mathlib/1.0 - Cache
  ai/1.0 - Cache
  engine/1.1 - Cache
```



Editable packages



SETUPTOOLS

Search

LINKS

- Home
- PyPI
- User guide
- Quickstart
- Supported Interfaces
- Package Discovery and Namespace Packages
- Dependencies Management in Setuptools
- Development Mode (a.k.a. "Editable Installs")**
- Entry Points
- Data Files Support

Development Mode (a.k.a. "Editable Installs")

When creating a Python project, developers usually want to implement and test changes iteratively, before cutting a release and preparing a distribution archive.

In normal circumstances this can be quite cumbersome and require the developers to manipulate the `PYTHONPATH` environment variable or to continuously re-build and re-install the project.

To facilitate iterative exploration and experimentation, setuptools allows users to instruct the Python interpreter and its import machinery to load the code under development directly from the project folder without having to copy the files to a different location in the disk. This means that changes in the Python source code can immediately take place without requiring a new installation.

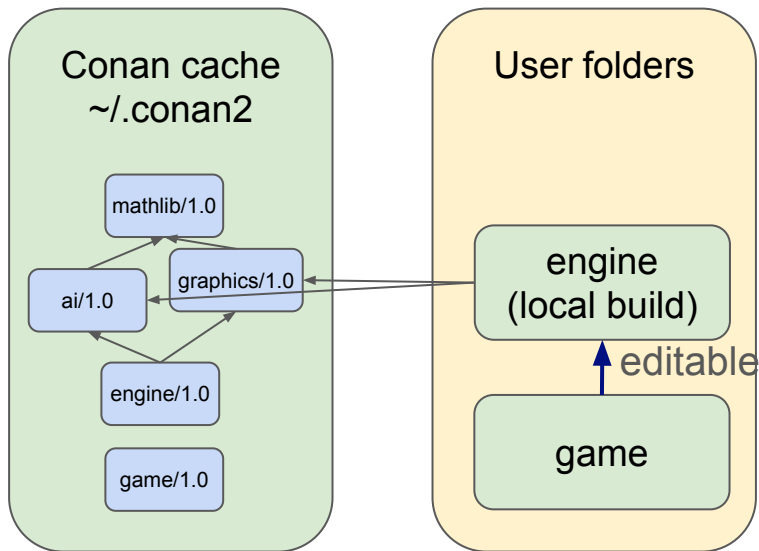
You can enter this "development mode" by performing an [editable installation](#) inside of a virtual environment, using pip's `-e/--editable` flag, as shown below:

```
$ cd your-python-project
$ python -m venv .venv
# Activate your environment with:
#   `source .venv/bin/activate` on Unix/macOS
# or  `.venv\Scripts\activate` on Windows

$ pip install --editable .

# Now you have access to your package
# as if it was installed in .venv
$ python -c "import your_python_project"
```

Editable packages

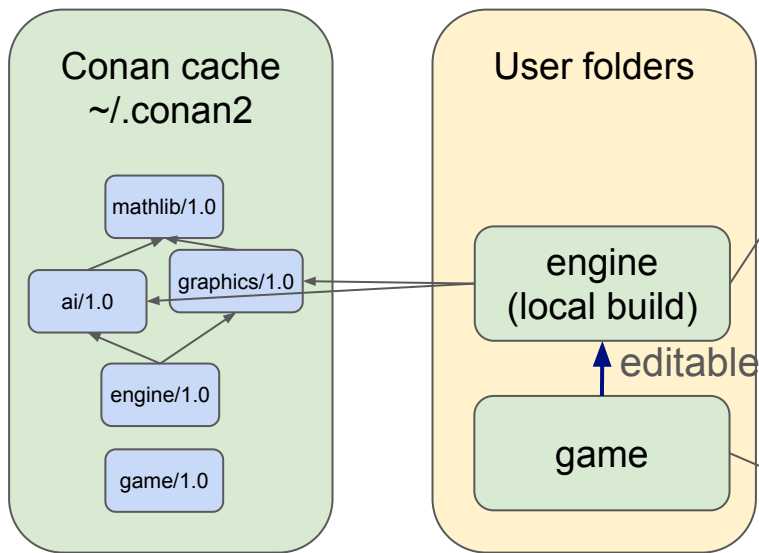


```
$ git clone git@.../engine.git
$ conan install
Requires
  mathlib/1.0 - Cache
  ai/1.1 - Cache
$ cmake --build # local build
$ conan editable add .
```

```
$ git clone git@game.git
$ conan install
Requires
  mathlib/1.0 - Cache
  ai/1.0 - Cache
  engine/1.1 - Editable
```



Still independent builds/projects



This screenshot shows the Visual Studio IDE with the `engine.cpp` file open. The code includes `<iostream>`, `"game.h"`, and `"engine.h"`. The `game` function is defined, and the `main` function calls `game()`. The output window shows the following error:

```
2> engine.cpp
2> C:\Users\Diego\vs\cpcon26\monow\engine\src\engine.cpp(3,18): error C1083: Cannot open include file:
3> "game.h" : No such file or directory
3> Build started: Project: game, Configuration: Debug x64
3> Building Custom Rule C:\Users\Diego\vs\cpcon26\monow\game\ChakeLists.txt
3> game.cpp
3> main.cpp
3> LINK : fatal error LNK1104: cannot open file '..\engine-build\Debug\engine.lib'
4> Build started: Project: ALL_BUILD, Configuration: Debug x64
4> Building Custom Rule C:\Users\Diego\vs\cpcon26\monow\ChakeLists.txt
4> Build 1: 2 failed, 0 up-to-date, 0 skipped
***** Build completed at 5:01 PM and took 02.988 seconds *****
```

This screenshot shows the Visual Studio IDE with the `game.cpp` file open. The code includes `<iostream>`, `"game.h"`, and `"engine.h"`. The `game` function is defined, and the `main` function calls `game()`. The output window shows the following error:

```
2> engine.cpp
2> C:\Users\Diego\vs\cpcon26\monow\engine\src\engine.cpp(3,18): error C1083: Cannot open include file:
3> "game.h" : No such file or directory
3> Build started: Project: game, Configuration: Debug x64
3> Building Custom Rule C:\Users\Diego\vs\cpcon26\monow\game\ChakeLists.txt
3> game.cpp
3> main.cpp
3> LINK : fatal error LNK1104: cannot open file '..\engine-build\Debug\engine.lib'
4> Build started: Project: ALL_BUILD, Configuration: Debug x64
4> Building Custom Rule C:\Users\Diego\vs\cpcon26\monow\ChakeLists.txt
4> Build 1: 2 failed, 0 up-to-date, 0 skipped
***** Build completed at 5:01 PM and took 02.988 seconds *****
```



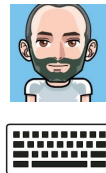
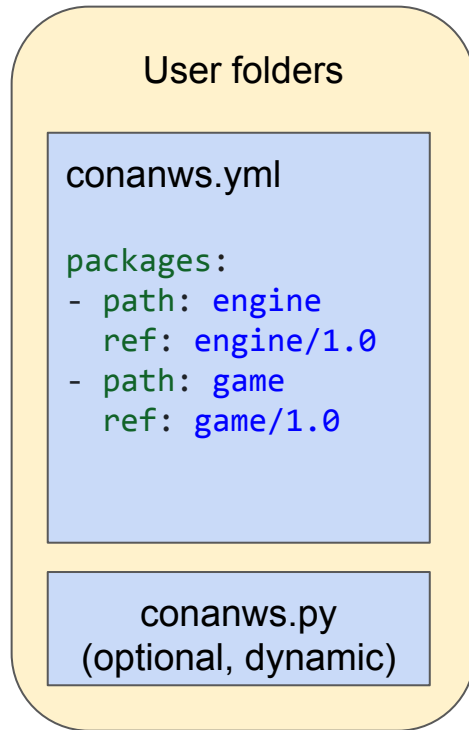
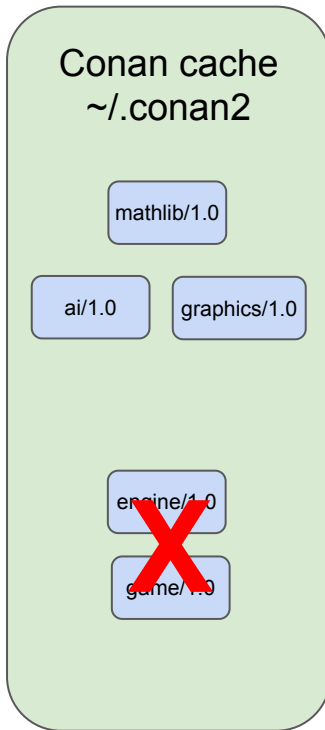
Outline

- Intro
- Example project
 - Package management basics
- The challenge: evolving packages
- CI at scale problem
- Workspace problem
 - **The orchestrated workspace**
 - The mono-repo/super-build workspace
 - The dynamic workspace
- Conclusions

Workspace

Definition: a set of locally editable packages

- Editable definition not global
- Dynamic: can add/remove packages
- Orchestrated:
 - Multi-repo
 - Mono-repo



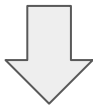
Demo: simple workspace open + build



SCM coordinates: traceability

conanfile.py

```
class aiRecipe(ConanFile):  
    name = "ai"  
    version = "1.0"  
    ...  
    def export(self):  
        git = Git(self, self.recipe_folder)  
        git.coordinates_to_conandata()
```



conandata.yml

```
scm:  
  commit: 98aee101ef132b2f7a63fbe99686991955da8817  
  url: git@github.com:memsharded/conanci_engine
```

Outline

- Intro
- Example project
 - Package management basics
- The challenge: evolving packages
- CI at scale problem
- Workspace problem
 - The orchestrated workspace
 - **The mono-repo/super-build workspace**
 - The dynamic workspace
- Conclusions

Mono-repo like workspace

conanws.yml

```
packages:  
- path: engine  
  ref: engine/1.0  
- path: game  
  ref: game/1.0
```

Represents the
aggregated
(collapsed)
workspace
super-build



conanws.py

```
class Ws(Workspace):  
    def root_conanfile(self):  
        return MyWs  
  
class MyWs(ConanFile):  
    settings = "os", "compiler", "build_type", "arch"  
    generators = "CMakeDeps", "CMakeToolchain"  
  
    def generate(self):  
        cmake = CMake(self)  
        cmake.configure()  
  
    def layout(self):  
        cmake_layout(self)
```

CMake aggregation via FetchContent

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.25)
project(myws CXX)

include(FetchContent)

function(add_project PACKAGE_NAME SUBFOLDER)
    FetchContent_Declare(${PACKAGE_NAME}
        SOURCE_DIR ${CMAKE_CURRENT_LIST_DIR}/${SUBFOLDER}
        SYSTEM OVERRIDE_FIND_PACKAGE)
    FetchContent_MakeAvailable(${PACKAGE_NAME})
endfunction()

add_compile_definitions(PKG_VERSION="WS_01")

add_project(engine engine)
add_library(engine::engine ALIAS engine) # project didn't
add_project(game game)
```

- Brings them together in the same CMake build
- find_package() works
- CPS from source/build tree request

Demo: mono-repo like build (super-build)



Outline

- Intro
- Example project
 - Package management basics
- The challenge: evolving packages
- CI at scale problem
- Workspace problem
 - Definition
 - The orchestrated workspace
 - The mono-repo/super-build workspace
 - **The dynamic workspace**
- Conclusions

Mono-repo like **dynamic** workspace

conanws.py

```
class Ws(Workspace):
    def packages(self):
        result = []
        for f in os.listdir(self.folder):
            if os.path.isdir(os.path.join(self.folder, f)):
                conanfile = self.load_conanfile(f)
                result.append({"path": f,
                              "ref": f"{conanfile.name}/{conanfile.version}"})
        return result

    def build_order(self, order):
        pkgs = " ".join([f'{it["ref"].name}:{it["folder"]}' for level in order for it in level])
        save(self, "build/conanws_build_order.cmake", f"set(CONAN_WS_BUILD_ORDER {pkgs})")

class MyWs(ConanFile):
    def generate(self):
        ..
        tc = CMakeToolchain(self)
        tc.preprocessor_definitions["PKG_VERSION"] = '"WS_0.1"'
        tc.generate()
```

Mono-repo like **dynamic** workspace CMake

```
cmake_minimum_required(VERSION 3.25)
project(myws CXX)
include(FetchContent)

function(add_project PACKAGE_NAME SUBFOLDER)
    FetchContent_Declare( ${PACKAGE_NAME}
        SOURCE_DIR ${CMAKE_CURRENT_LIST_DIR}/${SUBFOLDER}
        SYSTEM OVERRIDE_FIND_PACKAGE)
    FetchContent_MakeAvailable(${PACKAGE_NAME})
endfunction()

include(build/conanws_build_order.cmake)

foreach(pair ${CONAN_WS_BUILD_ORDER})
    string(FIND "${pair}" ":" pos)
    string(SUBSTRING "${pair}" 0 "${pos}" pkg)
    math(EXPR pos "${pos} + 1") # Skip the separator
    string(SUBSTRING "${pair}" "${pos}" -1 folder)

    add_project(${pkg} ${folder})
endforeach()
```

Demo: dynamic mono-repo like build (open + complete)



Development workspaces

- Principles
 - Traceability to source code and git coordinates
 - Being able to consume transparently from the source/build tree (conan layout() method)
 - CMake FetchContent
 - For others, like VS talk to us
 - CPS from the source/build tree
- Implementation with Conan2
 - **20 lines conanws.py + 20 lines super-CMakeLists.txt** code: **simple!**
 - No extra scripting necessary
 - Heterogeneous (not monolithic) workspaces
 - Mono-repo workspaces
 - Dynamic workspaces
 - **Proven:** Already used by Fortune 500 users even while incubating

Outline

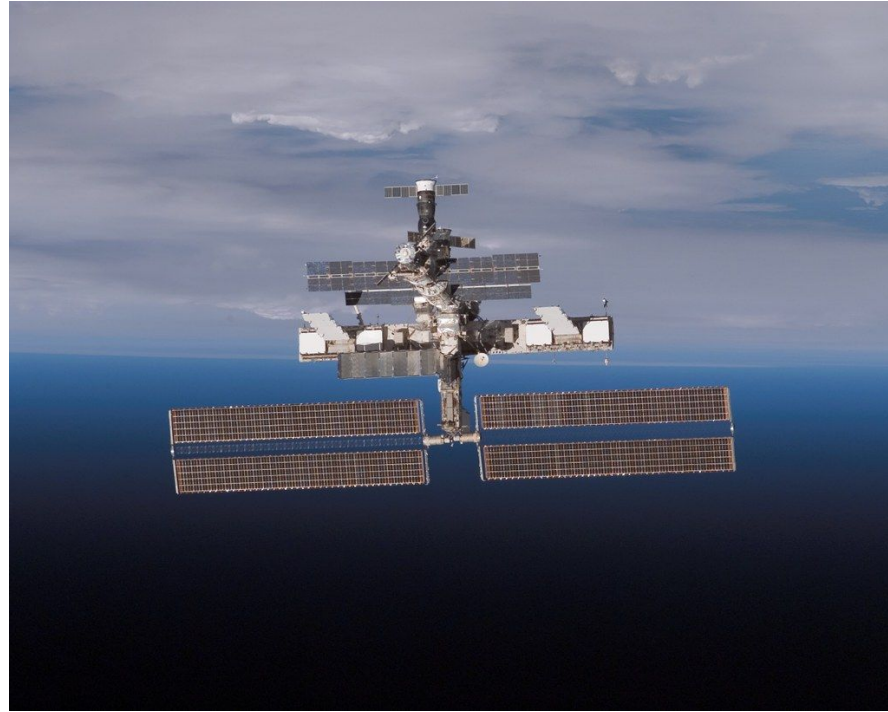
- Intro
- Example project
 - Package management basics
- The challenge: evolving packages
- CI at scale problem
- Workspace problem
 - The orchestrated workspace
 - The mono-repo/super-build workspace
 - The dynamic workspace
- **Conclusions**

Conclusions

- Dependency and package management is only the beginning
- Challenges:
 - Continuous Integration at scale
 - Bringing back components together for dev experience
- Principles:
 - Multi-repositories (“branch” in package server) and promotions (“merge” in package server)
 - Products pipelines bottom-up approach
 - Traceability from packages to sources
 - Build systems supporting dependencies to source/build trees
 - Dependency/package management tooling (graph build-order, workspace)
- Efficient, fast, safe and maintainable CI
- Convenient development workspaces (orchestrated and mono-repo like)

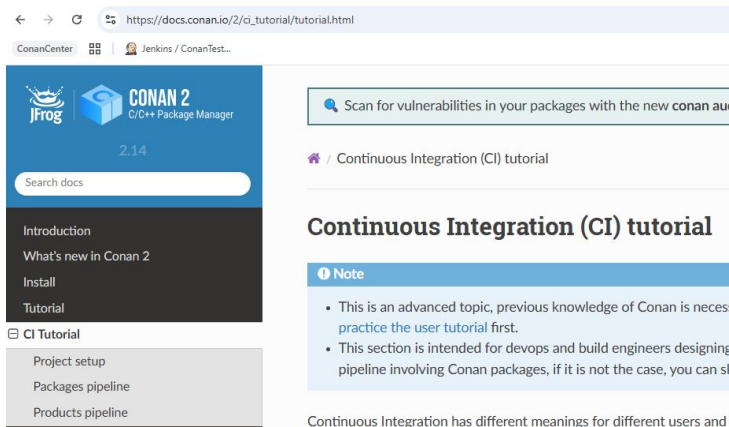
Component based paradigm

Seen by component
based
developers



Thank you!

Source code: https://github.com/memsharded/conan-ci_*



<https://docs.conan.io>



<https://conan.io>



<https://github.com/conan-io/conan>